

Input

At the beginning of the interaction, you are given an integer T ($1 \leq T \leq 10^4$), denoting the number of test cases.



Background. On August 9, Sensei attends the *Hidden Track. Blue Archive 3rd Anniversary Concert* at FEISHENG LIVEHOUSE in Shanghai with Seia on his shoulders. Unfortunately, having arrived late, they are stuck in a remote corner until Seia spots a mysterious pay-to-win passage to the second floor. She knows the only route through the crowd, but the riddle-loving Seia refuses to reveal it directly. Instead, Sensei must find the hidden track by following her query rules within the given time and query limits. Can you help him reach the second floor before the passage disappears?

Let $k = \lceil \log_2 n \rceil$. In each query, you choose two integers m and v , where $0 \leq m < 2^k$, and either $v = -1$ or $0 \leq v < n$. The integer m determines a set of vertices:

Here, $\&$ denotes the bitwise AND operation, and $\text{popcount}(x)$ denotes the number of 1-bits in the binary representation of x .

First, let $S = B(m)$.

The interactor then counts the edges of the hidden path with **exactly one endpoint** in S . Let this number be:

The interactor returns $c(S) \bmod 3$.

The interactor is non-adaptive: the hidden permutation for each test case is fixed before the interaction begins and does not change throughout the interaction.

Input

For each test case, the interactor first gives you an integer n ($1 \leq n \leq 10^3$). After you output the answer for the current test case, the interaction proceeds to the next test case, if any.

It is guaranteed that the sum of n over all test cases in one test file does not exceed 10^4 .

Interaction

For each test case, you may perform the following two types of operations. The number of queries must not exceed nk , with $k = \lceil \log_2 n \rceil$ as defined above.

Query: Output a line of the form `? m v`, where $0 \leq m < 2^k$, and either $v = -1$ or $0 \leq v < n$. Here m, v must be integers. Then read an integer $r \in \{0, 1, 2\}$, whose value is $c(S) \bmod 3$.

Answer: Output a line of the form `! q1 q2 ... qn`, where $(q_1, q_2, \dots, q_n)$ must be exactly equal to the hidden permutation $(p_1, p_2, \dots, p_n)$.

Only operations of the form `? count` toward the query limit. If the interactor returns -1 at any point, your program must terminate immediately. Continuing the interaction after receiving -1 may result in an arbitrary verdict.

After printing a query or the answer, do not forget to output the end of the line and flush the output. Otherwise, you will get the verdict *Idleness Limit Exceeded*. To do this, use: `fflush(stdout)` or `cout.flush()` in C++; `System.out.flush()` in Java; `flush(output)` in Pascal; `stdout.flush()` in Python; see the documentation for other languages.

Examples

Sample Input 1	Sample Output 1
2	? 1 -1
	? 1 0
3	? 1 1
1	? 1 2
1	! 1 0 2
0	
2	! 0
1	

Note

For the first test case, the hidden permutation is $p = (1, 0, 2)$. The path has two edges: $\{0, 1\}$ and $\{0, 2\}$.

All four queries use $m = 1$, so each query starts with $B(1) = \{1\}$:

- `? 1 -1`: The set remains $S = \{1\}$. Only edge $\{0, 1\}$ has exactly one endpoint in S , so the interactor returns 1.
- `? 1 0`: Add vertex 0, giving $S = \{0, 1\}$. Edge $\{0, 1\}$ has both endpoints in S , while edge $\{0, 2\}$ has exactly one. Thus, the interactor returns 1.
- `? 1 1`: Remove vertex 1, giving $S = \emptyset$. Neither edge has an endpoint in S , so the interactor returns 0.
- `? 1 2`: Add vertex 2, giving $S = \{1, 2\}$. Both edges have exactly one endpoint in S , so the interactor returns 2.

The program then outputs `! 1 0 2`. Here, $k = 2$, so at most $nk = 6$ queries are allowed. The example uses only four queries.

For the second test case, $n = 1$, so the only possible permutation is (0) . The program outputs `! 0` without making any queries.